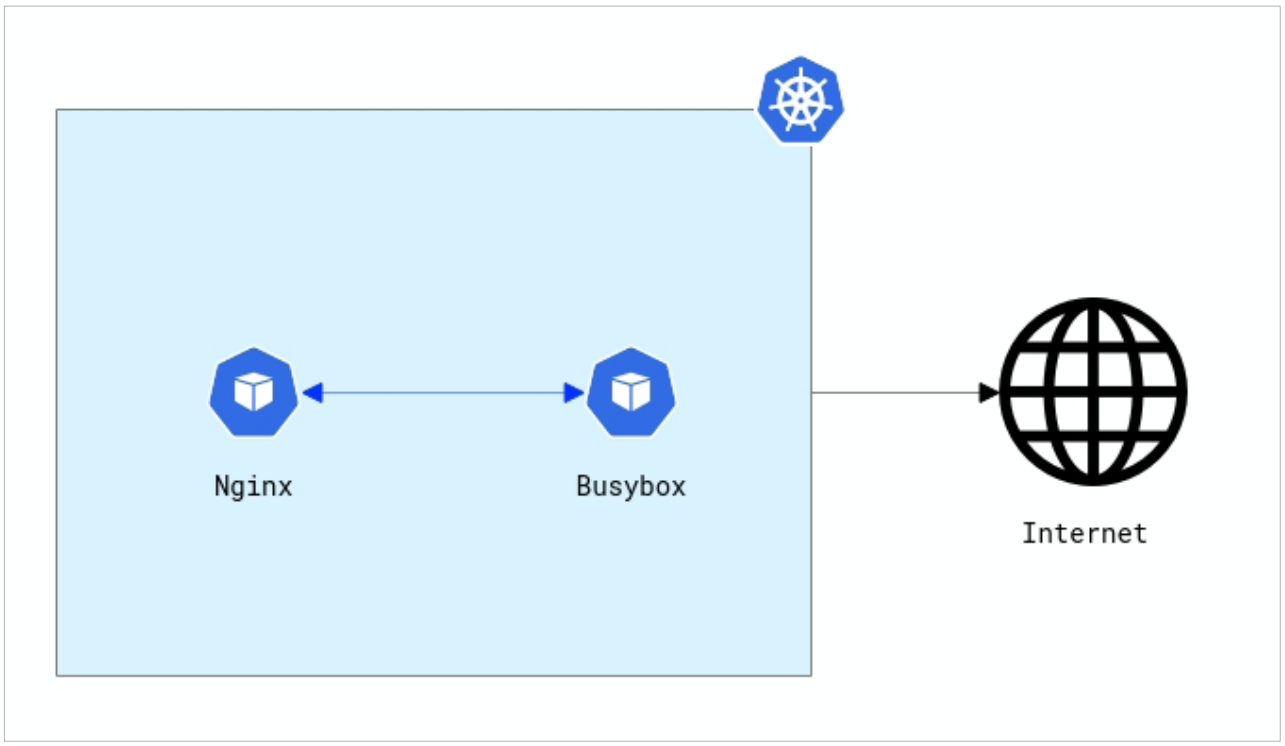

Figure 2: Two pods in a single name space

```
kns.kube-system
ksa.default.default
ksa.kube-node-lease.default
ksa.kube-public.default
ksa.kube-system.attachdetach-controller
ksa.kube-system.bootstrap-signer
ksa.kube-system.calico-kube-controllers
ksa.kube-system. calico-node
ksa.kube-system.coredns
ksa.kube-system. cronjob-controller
ksa.kube-system.calicoctl
ksa.kube-system. certificate-controller
ksa.kube-system.clusterrole-aggregation-controller
Ksa.kube-system.daemon-set-controller
ksa.kube-system.default
ksa.kube-system.deployment-controller
ksa.kube-system.disruption-controller
```

Cluster setup: For this demo, let us consider two pods, nginx and busybox, in a single name space (Figure 2).

Let us first create a name space called 'calico-demo':

```
$ kubectl create ns calico-demo
```

Now, we will create a pod with the image 'nginx':

```
$ kubectl -n calico-demo run nginx --image nginx
$ kubectl -n calico-demo label pod nginx app=nginx
$ kubectl -n calico-demo expose pod nginx --port=80
```

After this, we will create a pod with the 'busybox' image:

```
$ kubectl -n calico-demo run busybox --image=busybox
--restart=Never -- /bin/sh -c "tail -f /dev/null"
```

Default configuration: By default, both pods can communicate with each other. You can verify this by connecting to the busybox pod and communicating with the nginx pod, as follows:

```
$ kubectl -n calico-demo exec busybox -it -- sh
$ wget -q -T 5 nginx -O -
```

This will return the nginx welcome page.

Blocking all traffic: We can use network policies to block all ingress and egress traffic. To do this, let us create a global network policy in a file called *deny.yml:*

```
apiVersion: projectcalico.org/v3
kind: GlobalNetworkPolicy
metadata:
```